

Хакатон «ImageScope»

1 Задача

Создайте веб-приложение **ImageScope**. Команда самостоятельно выбирает один сайт-источник. Приложение находит размещённые на нём изображения, загружает их на сервер, анализирует визуальные характеристики, применяет выбранные фильтры и показывает результат в графическом интерфейсе.

Пример пользовательского сценария: команда выбирает каталог интернет-магазина, а пользователь запускает сбор, выбирает «резкость», «комикс» или другой фильтр и сравнивает исходные изображения с обработанными.

Обязательный минимум. Готовое решение должно образовывать единую цепочку: **интерфейс** → **FastAPI** → **парсинг страницы** → **анализ и фильтры NumPy/PyTorch** → **результат «до/после»**.

2 Функциональные требования

2.1 Получение и парсинг страницы

Приложение должно:

1. самостоятельно выбрать сайт с доступными без авторизации страницами и не менее 100 изображений;
2. сохранить адрес выбранного сайта в конфигурации или коде приложения и указать источник в README;
3. получить HTML страницы на backend с тайм-аутом;
4. корректно пропускать недоступные или повреждённые изображения и сообщать пользователю, что произошло.

Для предсказуемой демонстрации команда может дополнительно подготовить локальную тестовую HTML-страницу, однако основной сценарий должен собирать данные с выбранного командой сайта. Возможность менять источник через интерфейс не требуется.

2.2 Обработка изображений

Каждое успешно загруженное изображение необходимо преобразовать в RGB и представить как числовой массив. Для чтения допустимо использовать Pillow или OpenCV.

Статистика NumPy – обязательно:

- рассчитать ширину, высоту и соотношение сторон;
- рассчитать среднюю яркость и контраст (например, стандартное отклонение яркости);
- построить хотя бы одну агрегированную характеристику: RGB/gray-гистограмму, средний цвет или долю слишком светлых/тёмных пикселей.

Преобразование NumPy – обязательно: реализуйте хотя бы один фильтр операциями над массивами, а не только готовым методом Pillow/OpenCV. Например: grayscale, инверсия, коррекция каналов, пороговая обработка, гамма-коррекция, постеризация или пикселизация. Результатом должно быть новое изображение.

Свёртка PyTorch – обязательно:

- преобразовать изображение в тензор формата $C \times H \times W$;
- применить через `torch.nn.functional.conv2d` хотя бы одно ядро;
- получить и показать обработанное изображение, а не только число.

Подойдут размытие, повышение резкости, Собель, emboss или собственное ядро. Команда должна реализовать минимум **два видимых преобразования**: одно на NumPy и одно на PyTorch. Разрешается добавить цепочку фильтров, пресеты или регулируемую интенсивность. Загрузка больших весов **не требуется**.

2.3 Творческая идея и итоговый результат

Выберите понятную творческую концепцию: улучшение карточек товаров, ретро-стиль, комикс, выделение контуров, пиксель-арт, подготовка изображений для датасета или собственный вариант. Интерфейс должен показывать исходник и результат рядом. Статусы вроде **слишком тёмное** или **возможно размыто** можно оставить, но они не заменяют визуальную обработку. В README кратко объясните назначение фильтров и ожидаемый эффект.

3 Backend и API

Backend должен быть реализован на **FastAPI**. Минимальный контракт приведён ниже; названия и структуру можно расширять.

Метод	Маршрут	Назначение
GET	/health	Проверка готовности приложения
POST	/api/analyze	Запустить сбор и обработку, вернуть JSON-отчёт
GET	/	Открыть графический интерфейс (если он раздаётся FastAPI)

Запрос `/api/analyze` должен принимать название фильтра или пресета; можно добавить интенсивность и другие параметры. Ответ должен содержать источник, статистику, выбранные преобразования, ссылки или данные для показа оригиналов и результатов, а также список ошибок. Используйте Pydantic-модели. Один плохой файл не должен завершать весь запрос.

4 Графический интерфейс

Графический интерфейс обязателен. Подойдёт HTML/CSS/JavaScript с шаблонами Jinja2, React/Vue, Streamlit или Gradio. Если выбран Streamlit/Gradio, анализ всё равно должен выполняться через FastAPI API, а не в обход него.

В интерфейсе должны присутствовать:

- название или адрес выбранного командой сайта и кнопка запуска сбора;
- выбор фильтра/пресета; приветствуется настройка его интенсивности;
- состояние загрузки и понятное сообщение об ошибке;
- галерея с исходным и обработанным изображением в формате «до/после»;
- для каждого изображения: размер, яркость, контраст и применённые фильтры;
- хотя бы одна визуализация: гистограмма, цветовой индикатор, bar chart или сводные карточки.

Красивый дизайн приветствуется, но работоспособность и читаемость важнее сложной вёрстки.

5 Архитектура решения

Рекомендуется разделить код хотя бы на модули API, парсинга и анализа. Ограничьте число изображений (например, 10), размер ответа и время ожидания, чтобы приложение не зависало на большой странице.

6 Формат работы и результат

6.1 Команда

Продолжительность хакатона – **2–3 часа**. В команде может быть **от 1 до 3 человек**. Возможное распределение ролей: backend/API, парсинг и обработка изображений. Один участник может совмещать несколько ролей. Команда должна распределить задачи между участниками, а перед сдачей зафиксировать **фактический вклад каждого**, даже если первоначальный план изменился.

6.2 Что нужно сдать

- исходный код проекта;
- файл `README.md` с командой запуска и кратким описанием решения;
- `requirements.txt` либо `pyproject.toml`;
- ссылку на репозиторий или архив проекта;
- презентацию в формате PDF, PPTX или ссылку на онлайн-презентацию;
- работающую демонстрацию и питч продолжительностью до 3 минут.

Рекомендуемая команда запуска: `uvicorn app.main:app --reload`. Секреты, токены и большие веса моделей не должны попадать в репозиторий.

6.3 Презентация и вклад участников

Команда обязана подготовить короткую презентацию на **3–5 слайдов**. В ней должны быть: задача и идея продукта, архитектура решения, использованные NumPy/PyTorch-методы, творческая концепция, пример «до/после» и выводы.

Отдельный слайд или таблица должны показывать, **кто и что сделал**: имя участника, его зона ответственности и конкретный реализованный результат. Например: «Анна – FastAPI и модели ответа», «Иван – парсер и загрузка изображений», «Ольга – PyTorch-анализ, интерфейс и интеграция». На защите желательно, чтобы каждый участник кратко рассказал о своём вкладе.

7 Критерии оценки

Максимум – **100 баллов**. Неработающий обязательный компонент не получает баллы в соответствующей категории.

- | | |
|--|------------------|
| 1. Целостность продукта и демонстрация | 20 баллов |
| <ul style="list-style-type: none">• сценарий от запуска сбора до готового отчёта работает – 15;• понятная презентация, демонстрация и описание вклада команды – 5. | |
| 2. Парсинг и устойчивость | 15 баллов |
| <ul style="list-style-type: none">• извлечение изображений и разрешение относительных ссылок – 8;• тайм-ауты, пропуск ошибок, сообщения пользователю – 7. | |
| 3. Обработка изображений | 30 баллов |
| <ul style="list-style-type: none">• статистика и видимое преобразование NumPy – 8;• осмысленная свёртка PyTorch с визуальным результатом – 10;• корректное сравнение «до/после» и интеграция – 7;• творческая концепция и качество эффекта – 5. | |
| 4. FastAPI и качество backend | 15 баллов |
| <ul style="list-style-type: none">• рабочее API, валидация, структурированный JSON – 10;• разумная структура и читаемость кода – 5. | |
| 5. Графический интерфейс | 15 баллов |

- запуск, выбор фильтра, галерея «до/после» и метрики – 10;
- наглядная визуализация и удобство – 5.

6. Документация и воспроизводимость

5 баллов

Штраф до 20 баллов возможен, если заявленная технология присутствует только формально: например, PyTorch импортирован, но его результат нигде не используется, или интерфейс вызывает анализ напрямую, минуя обязательное API.